Edges

Study-Ready Notes

Compiled by Andrew Photinakis

March 22, 2026

Contents

1 The Physics and Definition of Edges **4**
1.1 Concept Overview . . . 4
1.2 Mathematical Formulation of Edge Detection . . . 5
1.2.1 The Image as a Function: $f(x, y)$ . . . 5
1.2.2 The Gradient (First Derivative) . . . 5
1.2.3 Zero-Crossing (Second Derivative) . . . 6
1.2.4 The Step Function (Ideal Edge Model) . . . 6
1.2.5 Example: Black-to-White Vertical Edge . . . 7
1.3 Intuition . . . 8
1.4 Practical Applications . . . 8

2 The Purpose of Edge Detection **8**
2.1 Concept Overview . . . 8
2.2 Mathematical Formulation . . . 9
2.3 Idea of a Mapping . . . 9
2.4 The Edge Function . . . 9
2.5 Memory Reduction . . . 10
2.6 Big Picture Insight . . . 10
2.7 Algorithm Process . . . 10
2.8 Practical Applications . . . 11
2.9 Limitations . . . 11

3 Image Gradients and Finite Differences **11**
3.1 Concept Overview . . . 11
3.2 Mathematical Formulation . . . 12
3.3 Algorithm Process . . . 13
3.4 Limitations . . . 14

4 Noise Sensitivity and Gaussian Smoothing **14**
4.1 Concept Overview . . . 14
4.2 Noise and Gaussian Smoothing . . . 14
4.3 Mathematical Formulation . . . 15
4.4 Algorithm Process . . . 16
4.5 Intuition . . . 16

5 Derivative of Gaussian Filters **16**
5.1 Concept Overview . . . 16
5.1.1 Key Mathematical Idea: Linearity and Associativity of Convolution . 17
5.1.2 Optimization Insight: Combining Operations . . . 17
5.1.3 Final Insight . . . 18
5.2 Mathematical Formulation . . . 18
5.3 Algorithm Process . . . 19
5.4 Limitations: The "One Size" Problem . . . 19

6 Scale-Space in Edge Detection **20**
6.1 Concept Overview . . . 20
6.2 Concept: What is "Scale"? . . . 21
6.2.1 Role of σ in Gaussian Blur . . . 21
6.3 Mathematical Formulation . . . 21
6.4 Algorithm Process . . . 22
6.5 Intuition: Brick Wall Example . . . 22
6.6 Limitations . . . 23

7 The Canny Edge Detection Pipeline **24**
7.1 Concept Overview . . . 24
7.2 Mathematical Formulation . . . 24
7.3 Algorithm Process . . . 26
7.4 Intuition . . . 27
7.5 Practical Applications . . . 27
7.6 Limitations . . . 27

8 Transitioning from Edges to Boundaries **27**
8.1 Concept Overview . . . 28
8.2 Concept Overview: The "Holistic" Leap . . . 28
8.3 Mathematical Formulation: Least Squares Line Fitting . . . 28
8.4 Algorithm Process: Minimizing Error . . . 29
8.5 Intuition: The "Rigid Ruler" Analogy . . . 29
8.6 Practical Application: Self-Driving Cars . . . 29
8.7 Limitations: Why this often fails . . . 29

9 Robust Regression and RANSAC **30**
9.1 Concept Overview . . . 30
9.2 Mathematical Formulation . . . 30

9.3 Algorithm Process 31
9.4 Intuition and Analogy 31
9.5 Practical Applications 32
9.6 Limitations and Edge Cases 32

10 The Hough Transform **32**
10.1 Concept Overview 32
10.2 Mathematical Formulation 33
10.3 Algorithm Process 33

1 The Physics and Definition of Edges

Edges are defined as local, sharp changes in image intensity (brightness) or color. In the real world, these changes arise from several physical phenomena. A **depth discontinuity** occurs when one object is in front of another, creating a sudden change in distance from the camera. A **surface reflectance discontinuity** happens when two adjacent surfaces have different colors or material properties, causing a change in how light is reflected. **Illumination discontinuities**, such as shadows, produce edges due to changes in lighting rather than object boundaries. Finally, changes in **surface orientation** affect how light reflects off a surface, leading to gradual or sharp intensity variations. These physical causes explain why edges are meaningful features in images.

An **edge** occurs when pixel values change rapidly over a small region of an image. In simpler terms, if you move from one point in an image to another and notice a quick jump in brightness (light to dark) or color, you have likely crossed an edge.

Analogy: Think of walking across a flat field and suddenly stepping onto a sidewalk curb. That sudden height change is similar to how an edge behaves in an image—a sharp transition rather than a gradual one.

Edges are important because they capture the **structure** of a scene. Instead of processing every pixel, a computer can focus on edges to understand object boundaries, shapes, and important visual features. In this sense, edges act like a **skeleton** of the image, preserving essential information while reducing unnecessary detail.

1.1 Concept Overview

1. Edge detection is how computer finds where things suddenly change in a photo
2. Such edges are most important parts of a picture b/c they show shapes of objects
3. Edge happens whenever brightness or color jumps quickly in one spot
4. While look like simple lines on screen, usually happen b/c of four things in real 3D world
   (a) **Depth Discontinuities (Occlusion):** A depth discontinuity happens when one object blocks another. For example, a person standing in front of a tree creates an edge at the boundary between them. This occurs because the foreground and background have different visual properties, leading to a sudden change in pixel values.
   (b) **Corners and Surface Orientation Changes:** Edges can form where surfaces change direction, such as at corners or folds. For example, the corner of a box creates an edge even if the entire box is the same color. This happens because different faces reflect light differently. **Surface Orientation** refers to the direction a surface is facing relative to a light source.
   (c) **Changes in Color or Material:** Edges occur when the actual material or color of a surface changes, such as black ink on white paper or the stripes of a zebra.

Reflectance is the property that describes how a surface reflects light, which determines its color.

(d) **Illumination Changes (Shadows):** Edges can also result from lighting differences. For example, when a shadow is cast on the ground, the surface itself remains the same, but the brightness suddenly decreases where the shadow begins, creating an edge.

1.2 Mathematical Formulation of Edge Detection

This section explains how a computer interprets an image as a mathematical object and uses calculus concepts to detect edges. The key idea is to transform a visual problem (finding object boundaries) into a problem of analyzing how a function changes.

1.2.1 The Image as a Function: $f(x, y)$

A digital image is not just a collection of pixels; it can be modeled as a mathematical function:

$$f(x, y)$$

Definition:

- x, y represent spatial coordinates (horizontal and vertical position in the image).
- $f(x, y)$ represents the **intensity** (brightness) at that point.

Interpretation: Think of the image as a **3D landscape**:

- The (x, y) plane is the ground.
- The value $f(x, y)$ is the **height** at that location.

Analogy:

- Dark pixels $\rightarrow$ low height (valleys, near 0)
- Bright pixels $\rightarrow$ high height (peaks, near 255)

This transformation allows us to use calculus tools to analyze images.

1.2.2 The Gradient (First Derivative)

Definition: The **gradient** measures how quickly the intensity changes:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

Key Idea: Edges occur where the intensity changes **rapidly**.
Magnitude of Gradient:

$$|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

Interpretation:

- Small gradient → flat region (no edge)
- Large gradient → steep change (edge)

Thresholding:

- A **threshold** is a cutoff value.
- If $|\nabla f| > T$, the pixel is classified as an edge.

Analogy: Walking on terrain:

- Flat ground → no edge
- Sudden cliff → strong edge

1.2.3 Zero-Crossing (Second Derivative)

Definition: The second derivative measures how the **rate of change itself** is changing. In images, this is often computed using the **Laplacian**:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Key Idea: Edges are located at points where the second derivative changes sign.
Zero-Crossing:

- Positive value → concave up
- Negative value → concave down
- Zero-crossing → exact center of the edge

Analogy: Climbing a hill:

- Before peak: slope increasing (positive curvature)
- After peak: slope decreasing (negative curvature)
- Peak point: where curvature changes sign → edge center

1.2.4 The Step Function (Ideal Edge Model)

Definition: A **step function** models a perfect edge:

$$f(x) = \begin{cases} I_1 & x < x_0 \\ I_2 & x \geq x_0 \end{cases}$$

Parameters:

- I_1: intensity before the edge

- I_2: intensity after the edge
- x_0: location of the edge

Interpretation: This represents an **instant jump** in brightness.

Reality Note: Real images are usually blurred, so edges are not perfectly sharp, but this model is useful for understanding.

1.2.5 Example: Black-to-White Vertical Edge

Consider an image defined as:

$$f(x, y) = \begin{cases} 0 & x < 10 \\ 255 & x \geq 10 \end{cases}$$

Step Function Representation:

- $I_1 = 0$ (black)
- $I_2 = 255$ (white)
- $x_0 = 10$

Edge Detection Methods:

1. **Gradient Method**
   - At $x = 10$, intensity jumps from 0 to 255
   - Gradient magnitude is very large:

$$255 - 0 = 255$$

   - Since this exceeds any reasonable threshold, the edge is detected.
2. **Zero-Crossing Method**
   - Second derivative shows:
     - Positive spike before $x = 10$
     - Negative spike after $x = 10$
   - The point where it crosses zero is:

$$x = 10$$

Final Insight:

- Gradient $\rightarrow$ detects **where change is large**
- Zero-crossing $\rightarrow$ detects **exact center of the edge**

Together, these methods allow computers to precisely identify object boundaries in images.

1.3 Intuition

1. Imagine in pitch black room with a small, bright flashlight
2. As you move light around, everything is dark until beam hits edge of a table
3. Suddenly, you see a bright line where table starts and darkness ends
4. Edge detection is just a computer doing exact same thing
5. It scans a picture looking for sudden jumps from dark to light
6. Instead of using eyes, it uses math to spot cliffs of brightness, which tell it where one object ends and another begins

1.4 Practical Applications

1. Identifying specific physical cause of an edge allows advanced CV algos to parse 3D geometry of a scene
2. Example, autonomous driving systems rely on depth discountinuities to segment pedestrians from static background, while relying on surface reflectance discontinuities to dtect painted lane markings

2 The Purpose of Edge Detection

The goal of edge detection is to extract meaningful structural information from an image while discarding unnecessary details. Instead of working with full color or intensity values, edge detection produces a sparse representation that highlights only significant transitions. This simplifies computation because fewer data points are processed. The resulting representation is often binary, where edges are marked and non-edges are ignored. This process preserves important shape and boundary information, making it useful for tasks like object recognition, segmentation, and scene understanding.

2.1 Concept Overview

1. Standard high-resolution digital image contains millions of pixels, representing overwhelming volume of data
2. Processing entire grid pixel by pixel is computationally expensive
3. Fundamental rationale for edge detection is massive **information compression**
4. By extracting only edges, intentionally discard vast amounts of visually irrelevant data, such as large uniform pathes of color, smooth lighting gradients, and flat textures
5. Output is a sparse, binary representation (an image of mostly 0s with a few 1s denoting edges) that is computationally efficient to process

2.2 Mathematical Formulation

Edge detection can be understood as a **mathematical transformation** that converts a rich image into a much simpler representation.

2.3 Idea of a Mapping

In mathematics, a **mapping** (or function) takes an input and produces an output.

- **Input domain:** A dense, multi-channel image
- **Output domain:** A sparse, binary image

Definitions:

- **Dense:** Almost every pixel contains meaningful information (like color values).
- **Multi-channel:** Each pixel stores multiple values (e.g., Red, Green, Blue).
- **Sparse:** Most values are zero, with only a few important points.
- **Binary:** Each pixel can only be one of two values (0 or 1).

Analogy: Think of a detailed photograph versus a simple sketch. The photograph contains all colors and textures (dense), while the sketch only keeps the important outlines (sparse).

2.4 The Edge Function

The transformation is defined as a function:

$$E(x, y) = \begin{cases} 1, & \text{if } (x, y) \text{ is an edge pixel} \\ 0, & \text{otherwise} \end{cases}$$

Explanation:

- (x, y) represents the position of a pixel in the image.
- $E(x, y)$ is the output at that pixel.
- If a sharp change (an edge) is detected at that point, we assign a value of **1**.
- If not, we assign **0**.

Key Idea: This function acts like a **filter** that keeps only important structural information (edges) and removes everything else.

2.5 Memory Reduction

A standard color image typically uses:

- 8 bits for Red
- 8 bits for Green
- 8 bits for Blue

$$\text{Total} = 24 \text{ bits per pixel}$$

After edge detection:

- Each pixel only needs **1 bit** (edge or no edge)

$$\text{New Total} = 1 \text{ bit per pixel}$$

Why this matters:

- **Memory efficiency:** The image becomes 24× smaller.
- **Speed:** Less data means faster processing.
- **Focus:** Only the most important structural features remain.

Analogy: This is like compressing a full-color movie into a line drawing storyboard. You lose detail, but keep the essential structure needed to understand the scene.

2.6 Big Picture Insight

This transformation is powerful because it:

- Converts complex visual data into a simple mathematical representation
- Highlights boundaries and shapes
- Enables higher-level tasks like object detection and recognition

In summary, edge detection is not just a visual trick—it is a **data reduction technique** that extracts meaningful structure from images using a simple binary function.

2.7 Algorithm Process

1. Process of creating sparse matrix involves thresholding
2. After calculating strength of intensity across image, algorithm applies a threshold to retain only most significant changes, converting continuous strength values into binary 1s and converting everything else to 0s

2.8 Practical Applications

1. Sparse representation is backbone of classical object recognition
2. Once image is reduced to structural skeleton, algos can perform shape analysis, line fitting, and template matching without being confused by color of an object or specific lighting conditions under which the photo was taken

2.9 Limitations

1. Aggressive discarding of data is double edged sword
2. By throwing away color and texture, algo binds itself to interior object features
3. A sparse edge map of a red apple and green apple of same size will look identical
4. If a task requires distinguishing objects based on texture or chromaticity, edge map alone is insufficient

3 Image Gradients and Finite Differences

Edge detection can be formulated mathematically using derivatives. In a continuous image, edges correspond to locations where the intensity function changes rapidly, which can be captured using the first derivative (gradient). For digital images, which are discrete, derivatives must be approximated using **finite differences**. A forward difference approximates the derivative using:

$$\frac{\partial I}{\partial x} \approx I(x+1, y) - I(x, y)$$

A central difference provides a more accurate approximation:

$$\frac{\partial I}{\partial x} \approx \frac{I(x+1, y) - I(x-1, y)}{2}$$

These approximations allow computation of gradients in both the x and y directions, forming the basis for detecting edges.

3.1 Concept Overview

1. To mathematically locate rapid change in image intensity, use calculus
2. In a 1D continuous signal $f(x)$, a sharp change (discontinuity) corresponds to a massive peak in first derivative $f'(x)$
3. B/c a digital image is a 2D function $I(x, y)$, extend this concept using **Gradient Operator**
4. Gradient is a 2D vector that points in direction of most rapid increase in image intensirty, and its magnitude dicates steepness of increase

Analogy: Think of driving a car:

- Position $\rightarrow f(x)$
- Speed $\rightarrow f'(x)$ (rate of change)

Definition: The *gradient* is a vector that:

- Points in the direction of the greatest increase in intensity
- Has a magnitude representing how steep that increase is

Analogy: Imagine standing on a mountain:

- The gradient direction tells you which way to walk to go uphill fastest
- The gradient magnitude tells you how steep the hill is

Conclusion: True derivatives cannot be computed; instead, we approximate them using **finite differences**.

Definition: Finite differences approximate derivatives by measuring changes between neighboring pixel values.

Analogy: Instead of measuring an exact slope at a point, you estimate it by looking at the difference between nearby points.

3.2 Mathematical Formulation

1. The continuous 2D gradient is defined as:

$$\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix}$$

2. For discrete images, we approximate using **central finite differences**:

$$\frac{\partial I}{\partial x} \approx \frac{I(x+1,y) - I(x-1,y)}{2}$$

$$\frac{\partial I}{\partial y} \approx \frac{I(x,y+1) - I(x,y-1)}{2}$$

3. **Explanation:**
   - $I(x+1,y) =$ pixel to the right
   - $I(x-1,y) =$ pixel to the left
   - The difference measures how intensity changes horizontally

4. These computations are efficiently implemented using **convolution**.

   Definition: Convolution ($*$) is an operation where a small matrix (called a kernel or filter) slides across the image to compute local changes.

   Horizontal gradient:

$$G_x = I * [-1 \;\; 0 \;\; 1]$$

   Vertical gradient:

$$G_y = I * \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

   Explanation:

   - G_x detects left-to-right intensity changes
   - G_y detects top-to-bottom intensity changes

   From these, we compute two key properties:

   1. Gradient Magnitude (Edge Strength)

$$\|\nabla I\| = \sqrt{G_x^2 + G_y^2}$$

   Meaning: Measures how strong the edge is.

   2. Gradient Orientation (Edge Direction)

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

   Meaning: Indicates the direction of the edge normal (perpendicular to the edge).

3.3 Algorithm Process

1. Convert the image to grayscale (if necessary).
2. Convolve the image with the horizontal kernel to compute G_x.
3. Convolve the image with the vertical kernel to compute G_y.
4. For each pixel (i, j):
   - Compute $\sqrt{G_x^2 + G_y^2}$ to get edge strength.
   - Compute $\tan^{-1}(G_y/G_x)$ to get edge direction.
5. Store results in gradient magnitude and orientation maps.

3.4 Limitations

Finite difference methods operate on very small neighborhoods (e.g., 1×3 windows). This causes a major issue:

- They respond to *any* rapid intensity change—including noise
- High-frequency noise produces false edges

4 Noise Sensitivity and Gaussian Smoothing

Differentiation is highly sensitive to noise because noise often contains high-frequency components. When computing derivatives, these high-frequency variations can be amplified, leading to false edge detections. To address this, a **Gaussian smoothing filter** is applied before computing gradients. This acts as a low-pass filter, reducing noise while preserving important structural information. The Gaussian function spreads intensity values in a controlled way, ensuring that the image becomes smoother and more stable for derivative computation.

4.1 Concept Overview

4.2 Noise and Gaussian Smoothing

- **Overview**

  This section explains why digital images are noisy and how Gaussian smoothing (blurring) is used to improve edge detection.

- **1. The Problem: Noise (Digital Static)**

  Digital cameras are not perfect. Images often contain noise, which appears as random visual artifacts.

  - **Thermal Noise:** Heat in the sensor creates random bright dots.
  - **Low Light Noise:** Insufficient light leads to grainy, "crunchy" images.
  - **Salt-and-Pepper Noise:** Transmission errors produce random black and white pixels.

- **2. Why Math Fails: The Noise Trap**

  Edge detection relies on identifying rapid changes in intensity (derivatives).

  - Noise consists of many small, rapid intensity changes.
  - These appear as false edges to mathematical operators.
  - As a result, real edges become obscured by excessive false detections.

- **3. The Solution: Gaussian Smoothing**

  To address noise, the image is smoothed before edge detection.

  - **Gaussian Filter:** A weighting function shaped like a bell curve.
  - **Process:** Each pixel is replaced by a weighted average of its neighbors.
  - **Effect:** Random noise is reduced because it cancels out during averaging.

- **4. Step-by-Step Process**

  1. Start with a noisy image.
  2. Apply Gaussian smoothing to reduce noise.
  3. Apply edge detection to the smoothed image.
  4. Extract only meaningful edges corresponding to real structures.

- **5. Intuition: Gravel Road Analogy**

  - Raw image: Driving with rigid wheels causes sensitivity to small bumps (noise).
  - Gaussian blur: Soft tires absorb small bumps, allowing only large bumps (true edges) to be detected.

- **6. Trade-Off: Loss of Precision**

  - Blurring reduces noise but also reduces sharpness.
  - Sharp edges become spread over multiple pixels.
  - The exact location of the edge becomes less precise.

- **Conclusion**

  Gaussian smoothing improves edge detection by removing noise, but introduces a trade-off between noise reduction and spatial accuracy.

4.3 Mathematical Formulation

Let $f(x)$ be our true, clean optical signal and $n(x)$ be high-frequency noise. Our observed image is

$$I(x) = f(x) + n(x).$$

When we apply the derivative operator:

$$\frac{d}{dx}I(x) = \frac{d}{dx}f(x) + \frac{d}{dx}n(x).$$

Because the spatial frequency of the noise $n(x)$ is vastly higher than the frequency of the true signal $f(x)$, the magnitude of its derivative $n'(x)$ explodes algebraically, entirely burying the true edge signal $f'(x)$.

4.4 Algorithm Process

To solve this mathematical instability, we must forcibly suppress the noise before computing the derivative.

Define a Gaussian low-pass filter:

$$g(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}.$$

Convolve the noisy input image with this Gaussian kernel to blur it:

$$I_{\text{smooth}} = I * g.$$

Compute the spatial derivative of the smoothed result:

$$\text{EdgeMap} = I_{\text{smooth}} * \frac{d}{dx}.$$

4.5 Intuition

1. Imagine feeling large, physical speed bumps on a gravel road while driving a car equipped with solid metal wheels
2. Every tiny pebble (noise) will violently shake the car, completely masking impact of speed bumps
3. Applying a Gaussian blur is mathematical equivalent of putting soft rubber tires on car
4. Rubber absorbs high-frequency impacts of tiny pebbles, allowing suspension to smoothly and clearly detect large, physical speed bumps (true edges)

5 Derivative of Gaussian Filters

Instead of performing smoothing and differentiation as separate steps, they can be combined into a single operation using the associative property of convolution. This leads to the concept of **Derivative of Gaussian (DoG)** filters. Mathematically:

$$\frac{d}{dx}(G * I) = \left(\frac{dG}{dx}\right) * I$$

This means the derivative of a smoothed image is equivalent to convolving the image with the derivative of a Gaussian kernel. Practical implementations include filters like the Sobel and Prewitt operators, which approximate this combined effect efficiently.

5.1 Concept Overview

In previous section, we established a strictly sequential two-step pipeline:

1. Blur image with a Gaussian kernel
2. Differentiate blurred image with a finite difference kernel

5.1.1 Key Mathematical Idea: Linearity and Associativity of Convolution

Definition (Convolution): Convolution is a mathematical operation that combines two functions (or matrices, in the case of images) to produce a third function. In image processing, it is used to apply filters (like blur or edge detection) to images.

Key Properties:

- **Linearity:** Scaling and adding inputs behaves predictably.
- **Associativity:** The grouping of operations does not change the result.

Because convolution is associative, we can rewrite the pipeline as:

$$(\text{Image} * \text{Gaussian}) * \text{Derivative} = \text{Image} * (\text{Gaussian} * \text{Derivative})$$

Analogy: Think of convolution like applying filters to a photo. Instead of applying a blur filter and then an edge filter separately, you can combine both filters into one and apply it once.

5.1.2 Optimization Insight: Combining Operations

Performing two separate convolutions requires:

- Two full passes over the image
- Higher computational cost (especially for large images)

Instead, we can:

- Pre-compute the **Derivative of the Gaussian**
- Use it as a **single filter**

Definition (Derivative of Gaussian): The derivative of a Gaussian is a function obtained by differentiating the Gaussian smoothing function. It captures how intensity changes while still incorporating smoothing.

Why this works:

- The Gaussian handles **noise reduction** (smoothing)
- The derivative detects **rate of change** (edges)
- Combining them allows both to happen simultaneously

5.1.3 Final Insight

By computing:

$$\text{Combined Filter} = \text{Gaussian} * \text{Derivative}$$

we can apply:

$$\text{Image} * \text{Combined Filter}$$

Result:

- Noise is reduced
- Gradients (edges) are computed
- Only **one convolution pass** is required

Intuition: Instead of doing "blur then detect edges," we design a smarter filter that does both at the same time. This is faster, cleaner, and more efficient—especially for large-scale image processing systems.

5.2 Mathematical Formulation

The continuous 2D Gaussian function is represented as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Definition (Gaussian Function): A Gaussian function is a smooth, bell-shaped surface used to blur images and reduce noise. The parameter σ (standard deviation) controls the amount of smoothing:

- Small $\sigma \rightarrow$ less blur (preserves detail)
- Large $\sigma \rightarrow$ more blur (removes noise)

Taking the Partial Derivative: To detect edges, we measure how intensity changes. This is done by taking the partial derivative with respect to x:

$$\frac{\partial G}{\partial x} = -\frac{x}{\sigma^2} G(x, y)$$

Key Insight:

- The Gaussian component smooths the image
- The derivative component detects change (edges)

Geometric Interpretation: When visualized in 3D, this function forms:

- One positive lobe
- One negative lobe

Analogy: This is similar to a slope detector placed on a smooth hill:

- Flat regions $\rightarrow$ near zero response
- Sharp transitions $\rightarrow$ strong positive or negative response

5.3 Algorithm Process

1. Compute the Kernel
   (a) Analytically compute Derivative of Gaussian matrix for chosen scale σ
   (b) This step converts the continuous mathematical formula into actual numerical values
2. Discretization
   (a) Truncate and discretize the continuous function into a finite grid (e.g. 3×3 or 5×5)
   (b) Discretization is process of converting a continuous function into a grid of values that a compute can use
   (c) Like converting a smooth curve into pixels on a screen
3. Convolution
   (a) Convolve raw input image exactly once with pre-computed matrix
   (b) Result
      i. Noise is reduced (due to Gaussian smoothing)
      ii. Image gradients are computed (due to derivative)
      iii. Output is **denoised gradient map**

Final Insight: This algorithm replaces a two-step process (blur + differentiate) with a single efficient operation, improving both performance and numerical stability.

5.4 Limitations: The "One Size" Problem

While this approach is computationally efficient, it introduces an important limitation: **fixed scale**.

The Problem

A small filter (such as a 3×3 Sobel filter) has a built-in level of smoothing determined by σ (the standard deviation of the Gaussian).

Definition (Fixed Scale): Fixed scale means the amount of smoothing is constant and cannot adapt to different levels of noise in an image.

If the image is extremely noisy (for example, a photo taken in a very dark environment), a small filter:

- Does not provide enough smoothing
- Fails to adequately remove noise

Analogy: This is like trying to clean a heavily stained surface with a small cloth—it simply does not have enough coverage to be effective.

The Trade-off

To handle high noise levels, we must increase the filter size (e.g., 15×15 or 31×31).

Effects of Larger Filters:

- **Improved Noise Reduction:** More smoothing power
- **Increased Computational Cost:** More calculations per pixel
- **Reduced Edge Precision:** Edges become thicker and less localized

Definition (Edge Localization): Edge localization refers to how accurately an algorithm can pinpoint the exact position of an edge.

Key Trade-off:

- Small filters $\rightarrow$ fast, sharp edges, poor noise handling
- Large filters $\rightarrow$ better noise handling, slower computation, blurrier edges

Final Insight: There is no single filter size that works optimally for all images. This limitation motivates more advanced techniques (such as multi-scale methods) that adapt to different noise levels dynamically.

6 Scale-Space in Edge Detection

The parameter σ in the Gaussian function controls the scale of smoothing. A small σ preserves fine details and detects high-frequency edges, such as textures. A large σ smooths out small variations and highlights only large, structural edges. This concept is known as **scale-space**, where analyzing an image at different scales reveals different levels of detail. Choosing the appropriate scale depends on the application and the type of features of interest.

6.1 Concept Overview

1. Edges do not possess a single, objective ground truth
2. Their validity depends entirely on scale of observation
3. When we apply a Gaussian blur to an image to combat noise, physical severity of that blur is strictly controlled by variance parameter, σ
4. The value of σ dictates the spatial width of a low-pass filter
5. This directly determines the scale of the edges that will survive the differentiation process
6. By manipulating σ, an engineer determines whether the algo will extract fine, high-frequency textural details or coarse, overarching structural boundaries.

6.2 Concept: What is "Scale"?

In the real world, edges exist at different sizes.

Fine vs. Coarse Scale

- **Fine Scale:** Tiny details, like the pores on a person's skin or the threads in a shirt.
- **Coarse Scale:** Large structures, like the outline of a person's head or the shape of the shirt.

6.2.1 Role of σ in Gaussian Blur

When we use a Gaussian blur to clean an image, the "strength" of that blur is determined by σ.

- **Small σ (Weak Blur):** Only the tiniest noise is removed. Fine details stay sharp.
- **Large σ (Strong Blur):** Everything but the biggest, most obvious shapes is washed away.

6.3 Mathematical Formulation

1. The scale space of an image is a continuous 3D volume defined by:

$$L(x, y, \sigma) = I(x, y) * G(x, y, \sigma)$$

   1. The Variables

   - $I(x, y)$ (**The Input**): The original, raw image.
     - x, y: Pixel coordinates (the "address" of a pixel).
   - $G(x, y, \sigma)$ (**The Gaussian Filter**): The blurring tool.
     - A 2D bell-shaped curve
     - Values are highest at the center and decrease outward
   - σ (**Scale Parameter**): Controls the width of the Gaussian.
     - Small σ: Thin curve $\rightarrow$ slight blur
     - Large σ: Wide curve $\rightarrow$ strong blur
   - $*$ (**Convolution**): The operation of applying the filter.
   - $L(x, y, \sigma)$ (**The Result**): A new image smoothed at scale σ

2. **Scale Space**

   (a) Scale space is a representation of an image at multiple levels of detail, created by progressively applying Gaussian blur with increasing σ.

   (b) As σ increases, the cutoff frequency of the low-pass filter drops, progressively destroying higher-frequency components in the spatial domain.

3. **Low-Pass Filter**
   (a) A low-pass filter removes high-frequency information (fine details and noise) while preserving low-frequency information (large structures).
   (b) **Analogy:** Imagine looking at an image through increasingly foggy glass:
      - Small $\sigma \to$ clear view with fine details
      - Large $\sigma \to$ only large shapes remain visible

6.4 Algorithm Process

There is no single "correct" σ. An algorithm must iterate through scale-space:

1. Define a set of scales (e.g., $\sigma \in \{1, 2, 4, 8\}$).
2. Generate the derivative of Gaussian filter for each scale.
3. Apply each filter to the image independently.
4. Analyze the resulting edge maps to find the structure at the desired semantic level.

Key Insight: Different scales reveal different types of features:

- Small scales capture fine details
- Large scales capture dominant structures

Final Insight: By analyzing multiple scales, we overcome the limitation of fixed filter size and obtain a more complete understanding of the image.

6.5 Intuition: Brick Wall Example

Think about looking at a brick wall from different distances:

- **Standing 1 inch away (Small σ):** You see every tiny crack in the brick, the grain of the sand in the mortar, and the rough texture of the surface. If you ran an edge detector here, it would find millions of tiny edges for every little bump.
- **Standing 20 feet away (Medium σ):** The tiny cracks and sand grains blur together. Now, the most obvious edges are the rectangular outlines of the individual bricks.
- **Standing 200 feet away (Large σ):** The lines between the bricks disappear. You might only see the edge of the entire wall against the sky.

Scale as a Detail Filter

1. **The "Dial" (σ)** Scale can be understood as a **detail filter** for a computer. Depending on how much blur is applied (controlled by σ), the computer perceives entirely different structures.
   - Turning it down (Small σ): Everything stays sharp and crisp.
   - Turning it up (Large σ): Everything becomes blurry and smooth.
2. **Fine Scale (Small σ = Tiny Blur)** When σ is small, very little smoothing is applied.
   - **What the computer sees:** Every tiny detail.
   - **Result:** Extremely sensitive edge detection.
   - **Downside:** Produces noise—too many small, insignificant edges.
3. **Coarse Scale (Large σ = Heavy Blur)** When σ is large, strong smoothing is applied.
   - **What the computer sees:** Only large, dominant structures.
   - **Result:** Fine details disappear; major edges remain.
   - **Upside:** Clean representation of overall structure.
4. **Summary: The Brick Wall Example**
   - Small σ (Magnifying Glass): Detects millions of tiny edges from surface texture.
   - Large σ (Far Distance View): Detects only the large brick boundaries.

6.6 Limitations

Navigating scale-space is computationally expensive. Furthermore, as σ increases, edge localization degrades severely.

Definition (Edge Localization): Edge localization refers to how precisely an algorithm can determine the exact position of an edge in an image.

Effects of Large σ

High levels of blurring introduce several problems:

- **Edge Bleeding:** Adjacent edges (such as the left and right sides of a thin line) begin to overlap and merge into a single, incorrect edge.
- **Loss of Detail:** Fine structures are smoothed out and disappear.
- **Corner Rounding:** Sharp corners become mathematically rounded due to heavy smoothing.

Analogy: This is similar to viewing a detailed drawing through thick fog—distinct lines blur together, and sharp features become smooth curves.

Key Trade-off:

- Larger $\sigma \rightarrow$ better noise reduction but worse localization
- Smaller $\sigma \rightarrow$ sharper edges but more noise

7 The Canny Edge Detection Pipeline

The Canny edge detector is a multi-step algorithm designed to produce clean and accurate edges. First, the image is smoothed using a Gaussian filter to reduce noise. Next, gradients in the x and y directions are computed to determine edge strength and direction. Then, **Non-Maximal Suppression (NMS)** is applied to thin edges by keeping only local maxima in the gradient direction. After that, **double-thresholding** classifies edges as strong, weak, or non-edges. Finally, **hysteresis** connects weak edges to strong ones if they are part of the same structure, producing a coherent edge map.

7.1 Concept Overview

1. Canny Edge Detector is most widely utilized edge detection algo in classical computer vision
2. It unites concepts of Gaussian smoothing, optimal gradient differentiation, and non-linear thresholding into a cohesive, multi-step pipeline
3. John Canny designed this pipeline to satisfy three specific mathematical requirements. Every step in the pipeline exists to solve one of these three problems:
   - **Robust Detection:** The algorithm must find as many real edges as possible and ignore as much noise as possible. It should not "see" edges where none exist (False Positives).
   - **Precise Localization:** The detected edge should be exactly in the center of the real physical boundary. As discussed earlier, blurring an image (to remove noise) typically "smears" edges. The goal is to recover the exact 1-pixel center of that smear.
   - **Single Response:** A single edge should not result in multiple parallel detections. If a boundary is thick and blurry, the algorithm must identify only the strongest response (the peak) and discard the rest.

7.2 Mathematical Formulation

This formula represents the **Double-Thresholding** stage of the Canny edge detection pipeline. Its purpose is to filter out noise while ensuring that faint, genuine edges are not lost.

The Formula Breakdown

The formula is a **piecewise function**, meaning the computer follows different rules depending on the value of the gradient magnitude.

$$Class(x, y) = \begin{cases} \text{Strong} & \text{if } M(x, y) \geq T_{high} \\ \text{Weak} & \text{if } T_{low} \leq M(x, y) < T_{high} \\ \text{Discard} & \text{if } M(x, y) < T_{low} \end{cases}$$

Variable Definitions

1. $M(x, y)$ — **The Gradient Magnitude**

- **What it is:** The "strength" of the edge at pixel (x, y).
- **How it's calculated:** Derived from horizontal (G_x) and vertical (G_y) intensity changes.
- **Intuition:**
  - High $M(x, y) \rightarrow$ sharp intensity change (strong edge)
  - Low $M(x, y) \rightarrow$ flat region (no edge)

2. T_{high} — **The High Threshold**

- **What it is:** Upper cutoff value.
- **Role:** Identifies the most confident, high-contrast edges.

3. T_{low} — **The Low Threshold**

- **What it is:** Lower cutoff value.
- **Role:** Filters out weak noise while preserving potential edges.

4. $Class(x, y)$ — **The Output Category** Each pixel is assigned to one of three categories:

- **Strong:** Definitive edges; always kept.
- **Weak:** Uncertain edges; kept only if connected to strong edges (via hysteresis).
- **Discard:** Noise or flat regions; removed immediately.

Why Use Two Thresholds?

Using a single threshold creates a trade-off:

- Too high → lose faint but real edges
- Too low → detect excessive noise

The double-threshold method introduces the **Weak** category:

- Weak edges are treated as "candidates"
- They are only kept if connected to strong edges

Key Insight: This approach balances sensitivity and noise reduction by delaying the final decision on uncertain edges.

Analogy: It is like saying: "This edge might be real, but I will only trust it if it is connected to something I am sure about."

7.3 Algorithm Process

1. **Smoothing (Noise Filter)** The image is first blurred using a Gaussian filter.
   - **Goal:** Reduce noise before analysis.
   - **Logic:** Removes high-frequency "salt and pepper" noise so that small artifacts are not mistaken for real edges.
2. **Gradients (Finding the Slope)** The Sobel operator is used to compute gradients in both the x and y directions.
   - **Result:** Each pixel has:
     - **Magnitude:** Strength of the edge
     - **Direction (θ):** Orientation of the edge (horizontal, vertical, diagonal)
3. **Non-Maximal Suppression (Thinning)** The algorithm refines thick, blurry edges into thin ones.
   - **Goal:** Achieve precise localization and a single response.
   - **Action:**
     - Compare each pixel with its neighbors along the gradient direction
     - Keep the pixel if it is a local maximum
     - Otherwise, suppress it (set to 0)
   - **Result:** Edges are reduced to 1-pixel-wide lines.
4. **Hysteresis Thresholding (Edge Connection)** Two thresholds are used to classify edges.

- **High Threshold:** Strong, reliable edges
- **Low Threshold:** Potential (weak) edges
- **Logic:**
  - Weak edges are kept only if connected to strong edges
  - Isolated weak edges are discarded
- **Result:** Continuous, clean edges with reduced noise

Final Insight: Each step in the pipeline addresses a specific problem:

- Noise reduction → Smoothing
- Edge detection → Gradients
- Edge refinement → Non-Max Suppression
- Edge connectivity → Hysteresis

7.4 Intuition

Imagine a mountain ridge obscured by thick fog (noise). You first apply a filter to clear the fog (Gaussian). Then you calculate the slope of the mountain (Gradients). You walk along the slope, and strictly paint a line only on the absolute highest peak of the ridge (Non-Maximal Suppression). Finally, to ensure the painted line does not have gaps, if the ridge drops slightly in height but still physically connects to the main peak, you keep painting (Hysteresis).

7.5 Practical Applications

The Canny pipeline is the standard preprocessing step for shape detection, contour tracing, and boundary extraction. It converts raw photographic data into clean, 1-pixel-wide line drawings.

7.6 Limitations

The algorithm is highly parameterized. It requires manual, hard-coded tuning of three distinct hyperparameters (σ, T_{low}, T_{high}). Finding a universal configuration that perfectly segments all objects across varying lighting conditions in a dynamic environment is practically impossible.

8 Transitioning from Edges to Boundaries

Detecting edges is only the first step toward understanding image structure. The next step is grouping these edges into meaningful boundaries, such as lines or shapes. Simple methods like least-squares line fitting attempt to model edges as straight lines. However,

these methods have limitations. For example, they struggle with vertical lines due to slope representation issues and are highly sensitive to outliers. These challenges motivate the need for more robust techniques.

8.1 Concept Overview

8.2 Concept Overview: The "Holistic" Leap

Once the Canny pipeline is finished, you are left with a Binary Edge Map.

Definition: Binary Edge Map: A digital grid where every pixel is either a 1 (edge) or a 0 (background). It looks like a white line drawing on a black background.

The Problem: You see a "square," but the computer only sees a Coordinate List—a giant spreadsheet of X and Y numbers like $(10, 5), (11, 5), (12, 5), \ldots$ It does not know these dots are connected or that they form a shape.

The Goal: We need to move to Parametric Boundaries.

Definition: Parametric Boundary: A mathematical equation that describes a shape using a few fixed numbers (parameters). For example, a line is defined by just two numbers: its slope (m) and its starting height (c).

8.3 Mathematical Formulation: Least Squares Line Fitting

The most common way to turn those dots into a line is Least Squares Optimization. We assume the dots form a straight line:

$$y = mx + c$$

The Variables:

- y and x: The vertical and horizontal positions of a pixel.
- m (Slope): The steepness of the line.
- c (Y-intercept): Where the line crosses the vertical axis (at $x = 0$).
- N: The total number of edge pixels we are looking at.
- (x_i, y_i): The specific coordinates of the "i-th" pixel in our list.
- E (Energy/Cost): This represents the "Total Error." It is a score of how badly our mathematical line fits the actual dots.

The Formula:

$$E = \frac{1}{N} \sum_{i=1}^{N} (y_i - mx_i - c)^2$$

What it's doing: For every single dot, we measure the vertical distance between the actual dot (y_i) and where our line predicts the dot should be ($mx_i + c$).

Why Square it? We square the distance so that a dot 5 pixels above the line and a dot 5 pixels below the line do not cancel each other out (both become a positive error of 25). Squaring also makes "big misses" much more expensive than "small misses."

8.4 Algorithm Process: Minimizing Error

To find the "best" line, the computer must find the specific m and c that make the total energy E as small as possible.

Partial Derivatives: In calculus, the lowest point of a curve occurs where the slope is zero. The computer calculates the partial derivative of E with respect to m and c. This determines how the error changes as we tilt or shift the line.

Set to Zero: We set both partial derivatives exactly to 0 to find the "bottom of the valley" where the error is minimized.

Solve: This creates a system of linear equations. Solving them algebraically provides a closed-form solution—a final, optimal pair of numbers for the slope (m) and intercept (c).

8.5 Intuition: The "Rigid Ruler" Analogy

Imagine you have a piece of graph paper covered in scattered dots. Least-squares fitting is like dropping a perfectly straight, rigid metal ruler onto the paper.

Because the ruler is rigid, it cannot bend to touch every dot. Instead, you slide the ruler up and down (c) and rotate its angle (m) until the total "empty space" between the edge of the ruler and all the dots is as small as physically possible. That final resting position is your "best fit" boundary.

8.6 Practical Application: Self-Driving Cars

In autonomous driving, cameras detect thousands of white paint pixels on the road (edges).

The car uses parametric fitting to turn those pixels into two straight lines representing the lane.

By identifying where these lines converge at a vanishing point, the car can determine the geometry of the road ahead and stay centered, even if the paint is partially faded.

8.7 Limitations: Why this often fails

While mathematically elegant, simple algebraic least-squares fitting has three "catastrophic" flaws in real-world environments:

- **The Vertical Line Failure:** The model $y = mx + c$ cannot represent a perfectly vertical line. As the line becomes vertical, the slope (m) approaches infinity, which causes computer memory to overflow or calculations to crash.
- **Catastrophic Outlier Sensitivity:** Because the error is squared, one single "bad" pixel (noise) far away from the true boundary exerts massive mathematical leverage. It acts like a heavy weight at the end of a long lever, violently pulling the line away from the true geometry.

- **Multiple Models:** Least squares assumes all points belong to exactly one line. If an image contains a square (four distinct boundaries), the math will attempt to draw a single, meaningless average line through the center of all four boundaries rather than identifying the four separate sides.

9 Robust Regression and RANSAC

To handle noise and outliers, robust regression methods are used. One widely used approach is **RANSAC (Random Sample Consensus)**. Instead of fitting a model to all data points, RANSAC repeatedly selects random subsets of points to estimate a model. It then evaluates how many points agree with this model (called **inliers**). The model with the highest number of inliers is chosen as the best representation. This approach is effective in situations where data contains significant noise or multiple structures.

9.1 Concept Overview

- Robust regression is a set of techniques designed to overcome the fundamental failures of traditional least-squares fitting.
- In many real-world computer vision scenarios, images contain "outliers"—data points caused by noise, shadows, or unrelated objects—that do not belong to the geometric shape being measured.
- While standard least-squares attempts to minimize the error of every single point simultaneously, it inadvertently allows a few distant outliers to "pull" the fitted line away from the true boundary.
- To solve this, the Random Sample Consensus (RANSAC) algorithm was developed.
- RANSAC operates on a "trial-and-error" philosophy: instead of using all data points to find one average model, it repeatedly selects the smallest possible subset of points to propose a candidate model and then searches for a consensus among the remaining data.

9.2 Mathematical Formulation

Success of RANSAC is governed by probabilistic equation that determines how many times the algorithm must "guess" to ensure it eventually picks a set of points free from outliers.

The formula is defined as:

$$p = 1 - (1 - w^k)^N$$

- p (Confidence Level): Probability that the algorithm will successfully find a valid model at least once. Usually, this is set very high, such as 0.99 (99% confidence).
- w (Inlier Ratio): The probability that any single randomly selected point is a valid "inlier" (part of the real shape) rather than noise. For example, if half the points are noise, $w = 0.5$.

- k (Minimum Points): The absolute minimum number of points required to define the geometric model. For a straight line, $k = 2$; for a circle, $k = 3$.
- N (Number of Iterations): The number of times the algorithm must repeat its random sampling.

Relationship: By rearranging this formula to solve for N, engineers can calculate exactly how many iterations are required based on the estimated amount of noise in the image. As the noise increases (as w gets smaller), the number of required iterations N grows exponentially to maintain the same confidence p.

9.3 Algorithm Process

- Hypothesize: The algorithm randomly picks the absolute minimum number of points (k) needed to form a shape. For a line, it picks exactly two random edge pixels.
- Model: It calculates the mathematical parameters (such as the slope m and intercept c) of the line that passes perfectly through those specific points.
- Verify: It tests this "hypothetically perfect" line against every other edge pixel in the image. For each pixel, it measures the perpendicular distance to the line. If a pixel's distance is smaller than a strict predefined threshold, it is labeled an Inlier; otherwise, it is an Outlier.
- Update: The algorithm counts the total number of inliers. If this "consensus set" is larger than any previously found, it saves this line as the current "best guess."
- Repeat: This cycle repeats for N iterations. Once finished, the algorithm takes the final "best" set of inliers and performs a standard least-squares fit on only those points to get a final, ultra-precise boundary, completely ignoring the noise pixels it identified.

9.4 Intuition and Analogy

- Imagine a crowded room where you are trying to find the "average" opinion on a topic, but you know 30% of the people are lying just to mess with the results. If you use a standard Least Squares approach, you average everyone's answer together. Because the liars give extreme, ridiculous answers, the final average is skewed and incorrect—it represents neither the truth nor the lies.
- RANSAC acts like a skeptical investigator. It randomly grabs two people from the crowd, asks their opinion, and then draws a line based only on what they said. It then goes to everyone else in the room and asks, "Who agrees with this specific line?"
- If the investigator accidentally picked a liar, almost no one will agree, and the guess is thrown out.
- Eventually, by pure chance, the investigator will pick two honest people. Suddenly, 70% of the room will agree with that line.

- This massive "Consensus" proves that those two people were telling the truth, and the 30% who disagree are easily identified as the outliers to be ignored.

9.5 Practical Applications

- RANSAC is a workhorse in modern technology. In Panoramic Image Stitching, it is used to align two photos. If a bird flies through the sky in one photo but not the other, the bird acts as a "moving outlier." RANSAC ignores the bird and focuses only on the static landmarks (like buildings) that provide a consistent consensus for the alignment.
- Similarly, in 3D Mapping (Lidar), RANSAC is used to find flat surfaces like floors and walls within a massive "point cloud" of millions of individual laser measurements.

9.6 Limitations and Edge Cases

- **Non-Determinism:** Because it relies on random sampling, there is a non-zero chance that the algorithm will "get unlucky" and never pick a pure set of inliers, resulting in a failed detection if N is too low.
- **Computational Explosion:** If an image is extremely noisy (e.g., 95% noise), the probability of picking two "good" points at once becomes so low that the number of iterations (N) required to find a consensus can reach millions, causing the computer to slow down significantly.
- **Threshold Sensitivity:** The "Inlier Distance Threshold" must be manually tuned. If it is too strict, real data is ignored; if it is too loose, the model remains corrupted by noise.

10 The Hough Transform

The Hough Transform is a global technique for detecting shapes in images. Instead of fitting models directly in image space, it transforms the problem into a parameter space where each edge point votes for possible shapes. For line detection, the normal form is used:

$$\rho = x \cos\theta + y \sin\theta$$

This representation avoids issues with vertical lines. Peaks in the parameter space correspond to detected lines. The method can be extended to detect circles by adding a radius parameter, and further generalized to detect arbitrary shapes using the **Generalized Hough Transform**. This voting-based approach is robust to noise and partial occlusion.

10.1 Concept Overview

- The Hough Transform is a robust, voting-based feature extraction technique used in computer vision to isolate specific shapes within an image.

- Unlike regression methods that attempt to fit a single model to all data points simultaneously, the Hough Transform allows every individual edge pixel to "vote" for all possible shapes it could potentially be a part of.
- This global approach makes it uniquely capable of detecting multiple instances of a shape—such as several different lines or circles—within a single, noisy image.
- By transforming spatial data from the standard Image Space (where we see pixels) into a Parameter Space (where we see mathematical models), the algorithm converts the difficult problem of shape detection into a simpler task of finding peak values in a grid.

10.2 Mathematical Formulation

- To represent lines, the Hough Transform avoids the traditional $y = mx + c$ formula because it cannot represent vertical lines, where the slope m becomes infinite. Instead, it utilizes the Normal Form of a line equation: $\rho = x\cos\theta + y\sin\theta$.
- In this equation, ρ (rho) represents the algebraic perpendicular distance from the origin to the line, and θ (theta) represents the angle of that perpendicular vector relative to the horizontal axis.
- A single point (x_i, y_i) in the image does not define one line; rather, it defines an infinite family of possible lines that pass through it. When plotted in the (ρ, θ) parameter space, this family of lines forms a continuous sinusoid curve.
- If multiple edge pixels lie on the same straight line in the image, their corresponding sinusoids in the parameter space will all intersect at a single point (ρ_0, θ_0), which identifies the parameters of that shared line.
- Lines: To avoid the infinite slope problem of $y = mx+c$, the Hough Transform uses the Normal Form of a line: $\rho = x\cos\theta + y\sin\theta$. Here, ρ is the perpendicular distance from the origin to the line, and θ is the angle of that perpendicular vector. In this parameter space, a single edge pixel (x_i, y_i) maps to a continuous sinusoid curve representing all possible lines passing through it.
- Circles: A circle is defined by $(x-a)^2 + (y-b)^2 = r^2$, where (a, b) is the center and r is the radius. If r is known, the parameter space is 2D (a, b). Every edge pixel in the image casts a vote in the shape of a circle in the parameter space. If r is unknown, the parameter space becomes a massive 3D volume (a, b, r).

10.3 Algorithm Process

- The execution of the Hough Transform follows a "quantize and vote" strategy.
- **Quantize Parameter Space:** The computer creates a discrete grid called an accumulator array. Think of this as a giant grid of "buckets." Every bucket represents a

specific combination of parameters (like a specific ρ and θ). Quantization is the process of breaking continuous values (like an angle) into these fixed, discrete "buckets."

- **Vote:** The algorithm iterates through every edge pixel (x, y) in the binary edge map. For that pixel, it calculates every possible shape it could belong to and adds a "vote" (+1) to the corresponding buckets in the accumulator array.
- **Find Maxima:** Once every pixel has finished voting, the computer looks for the buckets with the highest number of votes, known as local maxima. These peaks mathematically represent the most obvious and prominent shapes in the original image.

10.4 Practical Applications and Robustness

- A major strength of the Hough Transform is its robustness to occlusion (when an object is partially hidden). Because votes are collected globally, a line that is "dashed" or partially blocked by a tree will still have enough pixels to cast votes for the same (ρ, θ) bucket. This allows the computer to "see" the full boundary even if part of it is missing.
- Furthermore, the Generalized Hough Transform (GHT) allows the detection of shapes that don't have a mathematical formula, like a specific tool or a logo. It uses an R-table (a lookup table) based on the centroid (the geometric center) of the shape. Each boundary pixel uses its local orientation to vote for where the center of the object should be, allowing for the detection of highly complex, non-mathematical silhouettes.

10.5 Limitations

- The primary limitation is the curse of dimensionality. Every new parameter you add to a shape makes the accumulator array much larger. A line needs a 2D grid, but an unknown circle needs a 3D grid. If you wanted to find a random ellipse, you would need a 5D grid. As you add dimensions, the memory required grows exponentially, often making it too "heavy" for a computer to handle efficiently.
- Additionally, if the "buckets" (quantization) are too large, the detection becomes imprecise; if they are too small, the votes might be split across multiple buckets, making it impossible to find a clear winner.